

Misr International University

Faculty of Engineering

Department of Electronics and Communication Engineering

Course: EEC123 Computer Organization

Semester: Spring 2026

Design & Implementation of a 2-bit Arithmetic and Logic Unit (ALU)

Submitted by:

- Ali Ahmed Ismail – ID: [2024/00127]
- Yehia Saeid Mohamed – ID: [2024/09585]

Instructor:

Dr. Medhat Hussein

Dr. Lamiaa Abdel-Hamid

Teaching Assistant:

Eng. Mariam Yousry

Eng. Shady Medhat

Submission Date:

19/4/2026

Abstract

This project presents the design and implementation of a 2-bit Arithmetic and Logic Unit (ALU), which is a fundamental building block in modern processors. The ALU is capable of performing both arithmetic and logic operations based on control signals.

The design was implemented using multiplexers, logic gates, and an adder circuit. The system supports multiple arithmetic operations such as addition and subtraction and increment and decrement, as well as logic operations such as AND, OR, NOT and XOR.

Simulation was carried out using a digital design tool to verify correctness, and the hardware implementation was built using IC components on a breadboard. The results showed correct functionality for all operations.

This project demonstrates a practical understanding of digital design concepts and processor architecture.

Table of Contents

1. Introduction
2. Design Specification and Description
3. Circuit Diagram
4. Discussion
5. IC Pinouts

1. Introduction

In modern computing systems, the Arithmetic and Logic Unit (ALU) is one of the most important components of the processor. It is responsible for performing mathematical calculations and logical operations required to execute instructions.

This project focuses on designing and implementing a 2-bit ALU capable of performing both arithmetic and logic operations. The ALU operates on two 2-bit inputs and produces a result based on control signals.

The design uses fundamental digital components such as logic gates, multiplexers, and adders. The system was first tested using simulation software and then implemented physically using hardware components.

2. Design Specification and Detailed Description

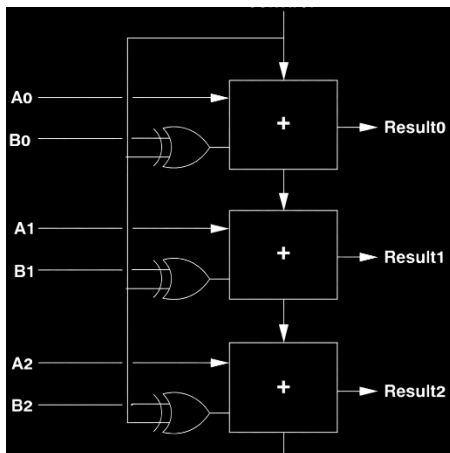
2.1 Specifications

- Number of bits: 2-bit
 - Inputs:
 - A (2-bit)
 - B (2-bit)
 - Control Signals: S2, S1, S0
 - Output:
 - Result (2-bit)
-

2.2 Operations

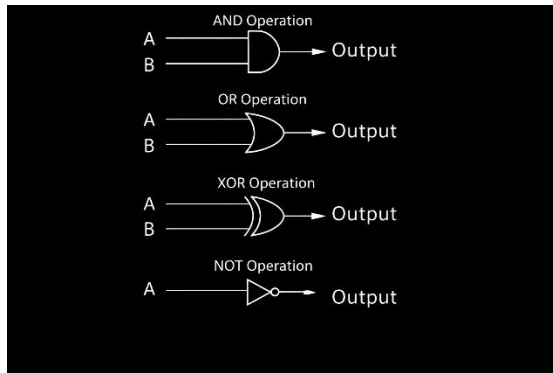
Arithmetic Operations:

1. $A + B$ (Addition)
2. $A - B$ (Subtraction using complement)
3. $A + 1$ (Increment)
4. $A - 1$ (Decrement)



Logic Operations:

1. $A \text{ AND } B$
2. $A \text{ OR } B$
3. $A \text{ XOR } B$
4. $\text{NOT } A$



2.3 Design Description

The ALU is divided into two main sections:

1. Arithmetic Unit:

The arithmetic part uses a full adder circuit to perform addition-based operations. Multiplexers are used to modify inputs to achieve subtraction, increment, and decrement operations.

- Subtraction is implemented using two's complement.
- Increment and decrement are achieved by controlling the input to the adder.

2. Logic Unit:

The logic section performs bitwise operations using logic gates:

- AND gate
- OR gate
- XOR gate
- NOT gate

3. Selection Mechanism:

Multiplexers are used to select the required operation based on control signals (S2, S1, S0).

- S0 selects between arithmetic and logic outputs.
 - S1 and S2 determine the specific operation inside each unit.
-

Control Signals and Operation Selection

S0 S1 S2 → Operation

0 0 0 → A + B

0 0 1 → A - B

0 1 0 → A + 1

0 1 1 → A - 1

1 0 0 → A AND B

1 0 1 → A OR B

1 1 0 → A XOR B

1 1 1 → NOT A

Truth Table (Representative Cases)

The following table shows representative test cases used to verify the correctness of the ALU operations.

Inputs: A, B (2-bit)

Example Cases:

S0	S1	S2	A	B	Operation	Result
0	0	0	01	01	A + B	10
0	0	1	10	01	A - B	01
0	1	0	01	XX	A + 1	10
0	1	1	10	XX	A - 1	01
1	0	0	11	01	AND	01
1	0	1	10	01	OR	11
1	1	0	10	01	XOR	11
1	1	1	10	XX	NOT A	01

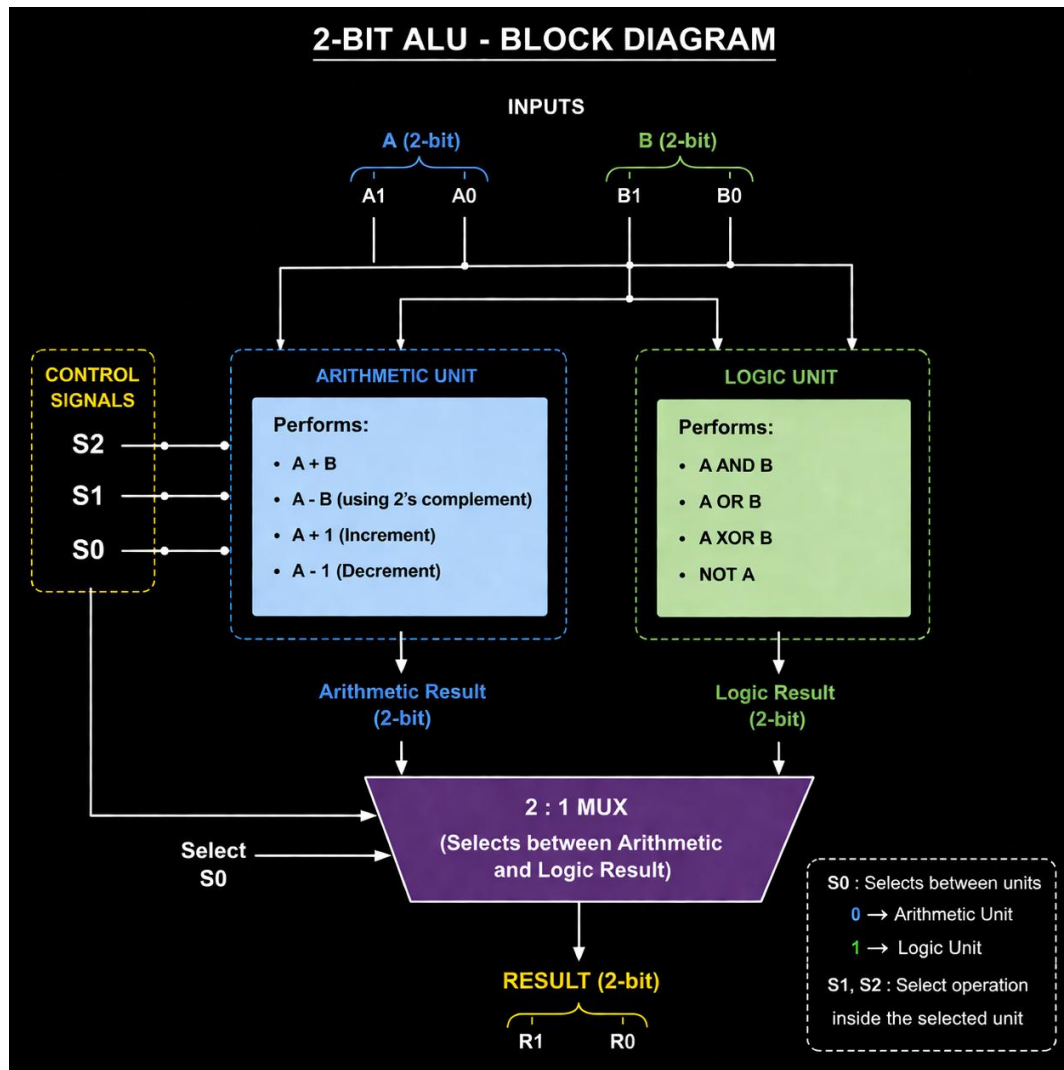


Figure 1: Block Diagram of the 2-bit ALU

The block diagram illustrates the overall structure of the ALU, including the arithmetic unit, logic unit, and the final multiplexer used to select the output based on control signals.

3. Circuit Diagram

3.1 Simulation Circuit:

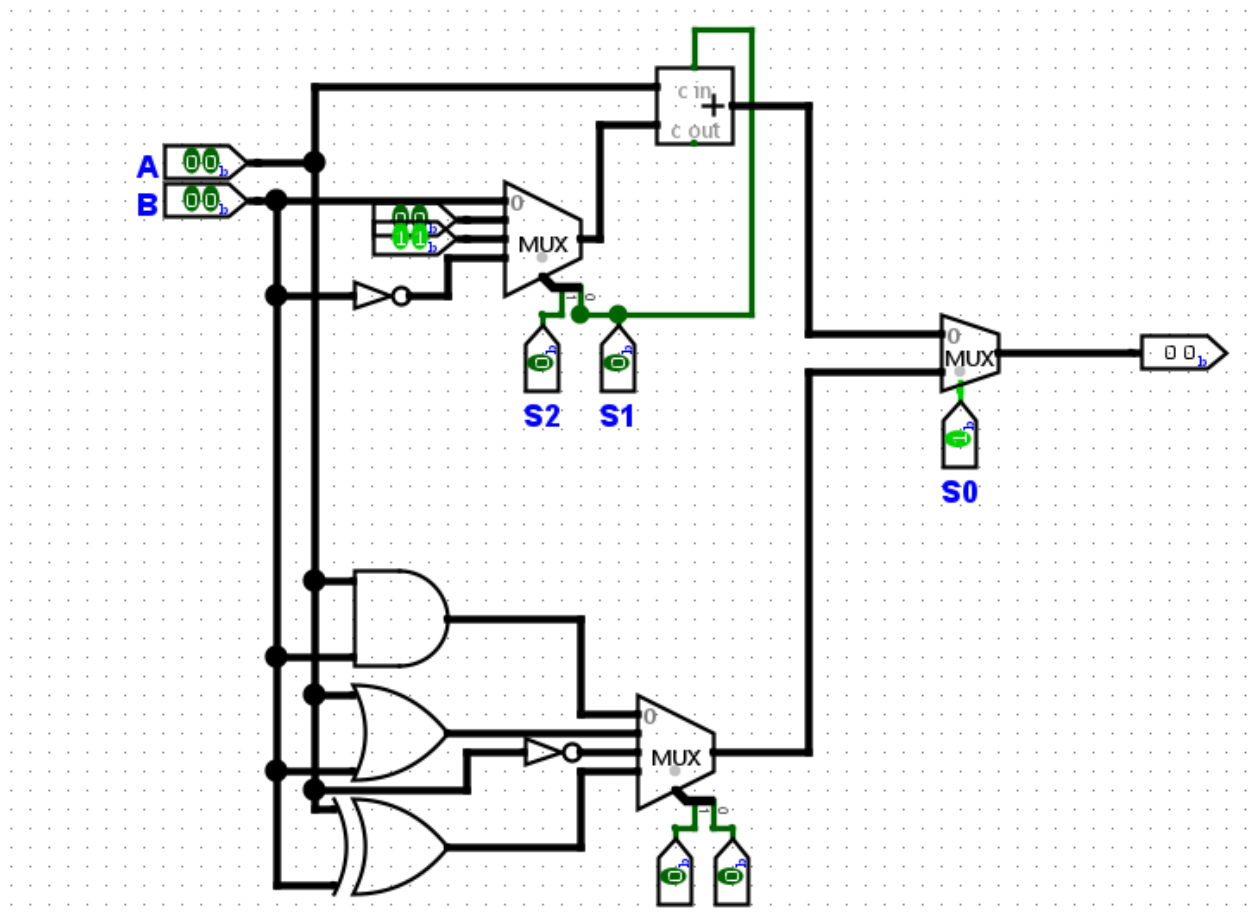


Figure 2: Simulation Circuit of the 2-bit ALU

3.2 Hardware Implementation:

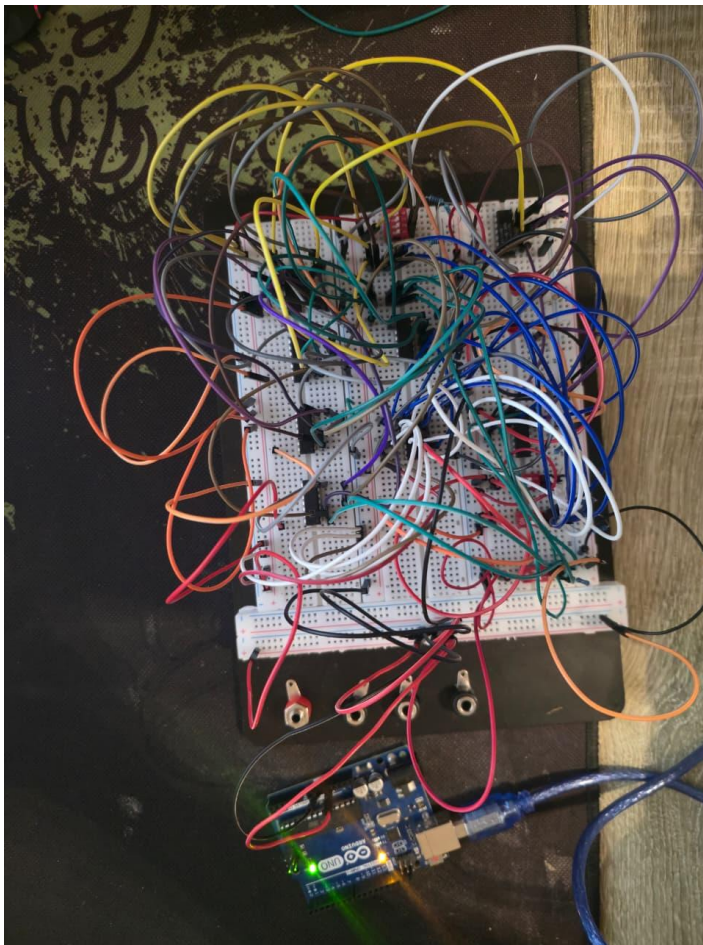


Figure3: Hardware Implementation on Breadboard

The simulation circuit consists of two main sections: an arithmetic unit and a logic unit.

Multiplexers are used to select between different operations based on control signals.

The final output is selected using a multiplexer controlled by S0.

4. Discussion

The project was successfully designed, simulated, and implemented using hardware components.

During simulation, all operations were verified and produced correct outputs for different input combinations. The hardware implementation also matched the simulation results.

One of the main challenges faced was the complexity of wiring on the breadboard, which required careful organization and debugging. Some incorrect connections initially caused wrong outputs, but these were resolved by testing each module individually.

Another challenge was ensuring proper synchronization between control signals and outputs. This was solved by verifying the logic step by step.

The use of multiplexers to control multiple operations significantly simplified the design.

Overall, the project provided a deeper understanding of how ALUs work and how different digital components are integrated to form a functional system.

4.1 Problem Faced

1. Floating Inputs and Enable Signal Misinterpretation:

During testing, incorrect or unstable outputs were observed due to floating inputs in some IC pins. Leaving certain pins unconnected caused undefined logic levels, resulting in random or constant outputs. Additionally, the enable pin was initially misunderstood, as it was active-low, meaning it required a logic 0 to operate correctly rather than logic 1.

2. Confusion in Final Multiplexer Connections:

Difficulty was encountered when connecting the output of the adder to the final multiplexer. Since the multiplexer had a single selector, it was initially unclear which input should be connected as the most significant or appropriate signal, leading to incorrect output selection.

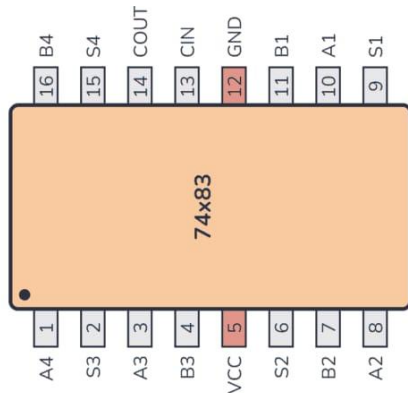
3. Wiring Complexity and Debugging Issues:

The complexity of the breadboard wiring caused multiple connection errors, leading to incorrect outputs. These issues were not fully resolved until the circuit was completely rewired, ensuring proper organization and accurate connections between all components.

5. IC Pinouts

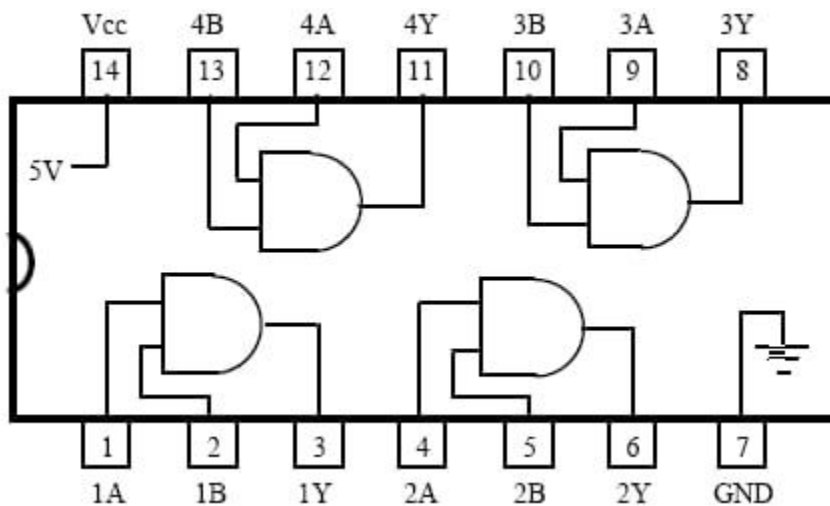
Example ICs Used:

74LS83 (4-bit Adder)



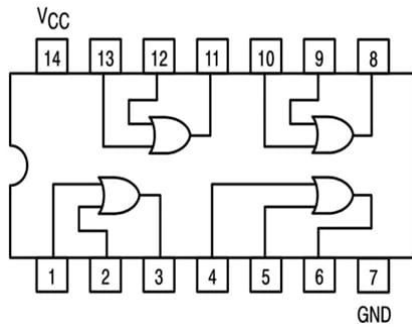
74LS08 (AND Gate)

- 4 AND gates inside IC



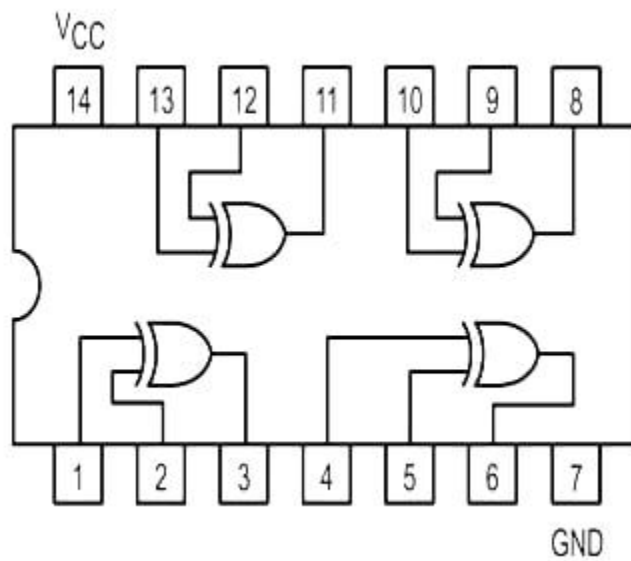
74LS32 (OR Gate)

- 4 OR gates



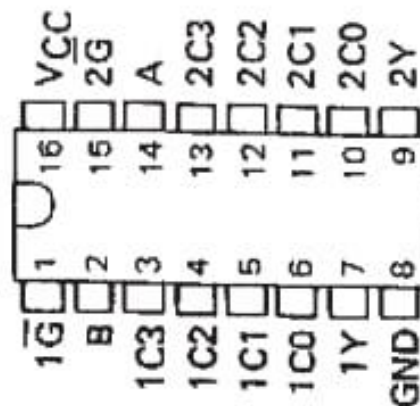
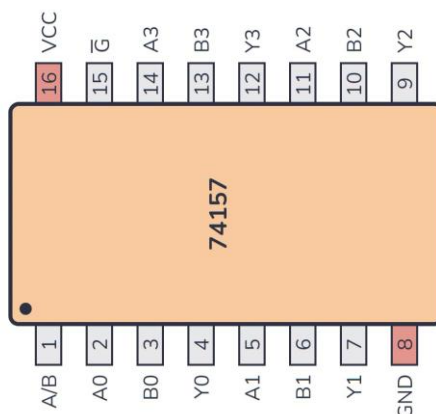
74LS86 (XOR Gate)

- 4 XOR gates



74LS157 / 74LS153 (Multiplexers)

- Used for selecting operations



Conclusion

This project successfully demonstrates the design and implementation of a basic ALU. It highlights how arithmetic and logic operations can be integrated using digital components. The combination of simulation and hardware implementation ensures both theoretical understanding and practical experience.